



Universidad Nacional de Colombia - sede Bogotá
Facultad de Ingeniería
Departamento de Sistemas e Industrial
Curso: Ingeniería de Software 1 (2016701)

Tutorial de construcción de aplicación

En este tutorial vamos a aprender a hacer el backend de una aplicación básica con conexión a base de datos en un contenedor Docker que va a permitir crear, consultar y eliminar usuarios mostrando el ID y nombre de usuario.

Para este tutorial se requiere la instalación de:

Lenguaje: Python

Framework: FastAPI

ORM: SQLAlchemy

Base de datos: PostgreSQL

Contenedor: Docker

Editor de código: Visual Studio Code

Vamos a hacer uso de esas tecnologías, puedes seguir el hipervínculo de cada una para acceder a su documentación o a su página oficial de instalación, para comprobar que ya se encuentran instaladas puedes abrir una terminal y escribir los siguientes comandos.

```
python --version
```

```
psql --version
```

```
docker --version
```

Estructura de la aplicación

```

backend_fastapi/
├── app/
│   ├── main.py
│   ├── database.py
│   ├── models.py
│   └── schemas.py
├── requirements.txt
├── Dockerfile
├── docker-compose.yml
└── .env
  
```

1. Editar el archivo Requirements.txt

Vamos a editar el archivo de requerimientos y dependencias necesarias, este se hace con el objetivo establecer las librerías necesarias y tenerlas en un archivo para su acceso y fácil instalación en otro dispositivo si es necesario.

```

requirements.txt
fastapi
uvicorn
sqlalchemy
psycopg2-binary
python-dotenv
  
```

2. Editar el archivo Dockerfile

Luego vamos a editar el archivo de Docker en el que establecemos una imagen base de Python, con esto se define el entorno de ejecución para que las dependencias necesarias de FastAPI se ejecuten de manera aislada, se van a instalar y ejecutar las librerías que establecimos en el paso anterior.

```

FROM python:3.12

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--reload"]
  
```

3. Editar el archivo Docker-compose.yml

Luego de eso vamos a configurar el archivo de composición del Docker, este se encarga de coordinar la sincronización de la base de datos con la API, el contenedor hace que ambos se comuniquen entre sí y asegura la disponibilidad de la base de datos antes del despliegue del servidor.

```
version: '3.9'
services:
  db:
    image: postgres:16
    container_name: postgres_fastapi
    restart: always

    environment:
      POSTGRES_USER: * EL USUARIO QUE TU CONFIGURASTE*
      POSTGRES_PASSWORD: * LA CONTRASEÑA QUE TU CONFIGURASTE*
      POSTGRES_DB: usuariosdb

    ports:
      - "5432:5432"

    volumes:
      - postgres_data:/var/lib/postgresql/data

  api:
    build: .
    container_name: fastapi_app
    restart: always

    depends_on:
      - db
    ports:
      - "8000:8000"
    volumes:
      - ./app

volumes:
  postgres_data:
```

4. Editar archivo .env

En este archivo se encuentra principalmente las variables de configuración bajo las cuales va a operar tu código, es importante separarlas del código principal para mantener la seguridad ante datos sensibles como el usuario o la contraseña, es importante que el usuario y contraseña coincidan con los del código del paso anterior y los que configuraste en PostgreSQL.

```
DB_USER=*EL USUARIO QUE TU CONFIGURASTE*
DB_PASSWORD=*LA CONTRASEÑA QUE TU CONFIGURASTE*
DB_NAME=usuariosdb
DB_HOST=db
DB_PORT=5432

DATABASE_URL=postgresql://${DB_USER}:${DB_PASSWORD}@${DB_HOST}:${DB_PORT}/${DB_NAME}
```

5. Configuración de la conexión de la base de datos

Este archivo se ubica en la carpeta principal del proyecto y usa SQLAlchemy para establecer la conexión principal con la herramienta de base de datos, acá defines la clase Base para crear las tablas que vas a usar en tu aplicación.

```
app/database.py

from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
from dotenv import load_dotenv
import os

load_dotenv()

DATABASE_URL = os.getenv("DATABASE_URL")
engine = create_engine(DATABASE_URL)

SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)

Base = declarative_base()
```

6. Crear entidad Usuario

En este archivo defines el modelo para la tabla o clase usuario, a la cual le asignas un id, un nombre y un correo, siendo el ID la llave primaria incremental que lo va a identificar.

app/models.py

```
from sqlalchemy import Column, Integer, String
from .database import Base

class Usuario(Base):

    __tablename__ = "usuarios"

    id = Column(Integer, primary_key=True, index=True)
    nombre = Column(String, nullable=False)
    correo = Column(String, unique=True, nullable=False)
    Base = declarative_base()
```

7. Editar el archivo Schemas

Este archivo permite que quien crea el usuario ingrese el nombre y correo correspondiente en formato de cadena de texto, automáticamente la respuesta que se crea es el usuario creado con esos dos atributos más el ID incremental que definimos previamente.

app/schemas.py

```
from pydantic import BaseModel

class UsuarioCreate(BaseModel):
    nombre: str
    correo: str

class UsuarioResponse(BaseModel):
    id: int
    nombre: str
    correo: str

class Config:
    from_attributes = True
```

8. Crear la FastAPI principal

En este archivo definimos la lógica central del proyecto, inicialmente se debe mostrar un mensaje al cargar el servidor, en este mismo se establece la lógica que le permite al usuario crear y eliminar usuarios y atributos almacenados en la base de datos previamente establecida.

```
app/main.py
```

```
from fastapi import FastAPI, HTTPException
app = FastAPI()
```

```
# Conexion temporal
```

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

```
@app.get("/")
def inicio():
    return {"mensaje": "Hola Mundo FastAPI"}
```

```
# Crear usuario
```

```
@app.post("/usuarios")
def crear_usuario(usuario: UsuarioCreate):
```

```
    db: Session = next(get_db())
```

```
    nuevo_usuario = Usuario(
        nombre=usuario.nombre,
        correo=usuario.correo
    )
```

```
    db.add(nuevo_usuario)
    db.commit()
    db.refresh(nuevo_usuario)
```

```
    return nuevo_usuario
```

```
# Ver usuarios
```

```
@app.get("/usuarios")
def listar_usuarios():
```

```
    db: Session = next(get_db())
```

```
    usuarios = db.query(Usuario).all()
```

```
    return usuarios
```

```
)
```

```

# Eliminar usuario
@app.delete("/usuarios/{usuario_id}")
def eliminar_usuario(usuario_id: int):

    db: Session = next(get_db())
    usuario = db.query(Usuario).filter(Usuario.id == usuario_id).first()

    if not usuario:
        raise HTTPException(status_code=404, detail="Usuario no encontrado")

    db.delete(usuario)
    db.commit()

    return {
        "mensaje": f"Usuario con ID {usuario_id} eliminado correctamente"
    }

```

9. Ejecutar el proyecto

Luego debes abrir una consola dentro del archivo de la aplicación en Visual Studio Code e ingresar el siguiente comando, el cual va a permitir levantar el contenedor de Docker y construir la imagen con las actualizaciones de código, iniciar FastAPI y conectar la base de datos y el backend, luego deberías ver un mensaje de carga de aplicación completa.

```
docker compose up --build
```

10. Prueba de mensaje

Al abrir el siguiente enlace en un navegador el cual contiene el host local cargado en el servidor 8000 deberías ver el mensaje que definimos en el archivo main.

<http://localhost:8000>

11. Prueba con Postman

Si ingresas el siguiente enlace puedes realizar la prueba de los endpoints sin necesidad de Postman, ya que es una utilidad que posee FastAPI.

<http://localhost:8000/docs>

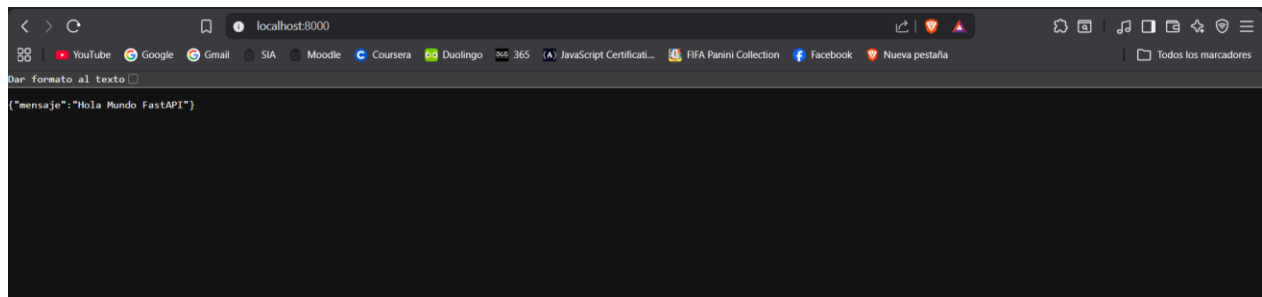
Para realizar la prueba en POSTMAN ingresas a esta aplicación, cambias en el menú de utilidades el método GET por POST y pegas el enlace que se encuentra a continuación, luego seleccionas la opción Body y luego raw, seleccionas JSON para poder crear, ver y eliminar los usuarios que están en la base de datos.

Si cargas directamente el enlace en un navegador corriente te va a mostrar un error porque generalmente solo hacen peticiones de tipo GET, mientras que POSTMAN si puede realizar como su nombre lo indica peticiones de métodos POST.

<http://localhost:8000/usuarios>

Y listo :D, ya creaste una aplicación backend conectada a una base de datos básica. El resto te lo dejo a tu imaginación, con esto puedes crear grandes aplicaciones para solucionar distintos problemas o necesidades.

PRUEBAS FINALES



Cuando abras el link del servidor local vas a darte cuenta que se ve el mensaje que habíamos escrito

POST /usuarios/ Crear Usuario

Parameters

Name	Description
nombre * required string (query)	Hugo Rodallega
correo * required string (query)	hugol@correo.com

Execute Clear

Responses

Cuando pruebas el endpoint vas a tener que ingresar el String para el nombre y el String para el correo del usuario

Server response

Code	Details
200	Response body <pre>{ "id": 2, "nombre": "Hugo Rodallega", "correo": "hugol@correo.com" }</pre> Response headers <pre>content-length: 62 content-type: application/json date: Fri, 05 Jun 2020 15:41:18 GMT server: uvicorn</pre>

El código 200 significa que la solicitud se ejecutó correctamente y se ha creado el usuario con los valores que ingresaste, además de un ID progresivo.

localhost:8000/usuarios/

[[{"id":1,"nombre":"Hugo Rodallega","correo":"hugol@correo.com"}, {"id":2,"nombre":"Nahuel Bustos","correo":"lautor@correo.com"}, {"id":3,"nombre":"Emanuel Olivera","correo":"turro@correo.com"}]]

Como puedes ver, al realizar el método get puedes ver los usuarios que has creado con sus atributos correspondientes.